SRM UNIVERSITY AP
Andhra Pradesh

PROJECT REPORT

EXCESS-3 ADDER

SUBMITTED TO

Dr. Raghavendra Shukla

BY

Shaikh Jaheed	- AP24110020001
Ch. Pradeep	- AP24110020016
Dhananjaya Sahu	- AP24110020041
Pratik Mahata	- AP24110020050

ABSTRACT

This project presents the design and implementation of an Excess-3 (XS-3) adder using Verilog HDL. Excess-3 is a self-complementing binary-coded decimal (BCD) representation widely used in digital arithmetic systems to simplify decimal operations. The objective of this project is to develop a hardware-efficient XS-3 adder capable of performing accurate decimal addition while maintaining correct Excess-3 encoding throughout the process. The design includes modules for binary addition, correction logic, and re-encoding to ensure valid XS-3 outputs for all input combinations. Simulation and functional verification were carried out using industry-standard EDA tools to validate correctness, timing behavior, and resource utilization. The results demonstrate that the proposed XS-3 adder performs reliable decimal arithmetic and can be integrated into larger digital systems such as calculators, digital counters, and arithmetic units.

Excess-3 Adder

The Excess-3 adder is a type of binary-coded decimal (BCD) adder used in digital systems to perform arithmetic operat-ions on decimal numbers. It is a digital circuit that adds two numbers already in Excess-3 code (a non-weighted decimal code where '3' is added to each BCD digit.

Eg: Let's take decimal 2, in binary 2 = 0010

Now we need to add decimal 3 to it for writing in Excess-3 code

Binary of decimal 3 = 0011

Add binary 2 and binary 3, 0010 + 0011= 0101.

Theory Involved: Every decimal number is written first in Excess-3 and then added, the output is also in Excess-3 code.

Let us take an example, 5+4 = 9

Step-1: Representing the decimal numbers in binary and then converting to Excess-3
5 - 0101 - 1000
4 - 0100 - 0111

Step-2: Adding the two Excess numbers
1000(5) + 0111(4) = 1111
There is no carry

Step-3: Check for carry, if there is no carry present then subtract 0011(3) from the sum.
1111 – 0011 = 1100
1100, this is the Excess-3 code for decimal 9

If a carry is present then you need to add 0011 to the 4-bit sum.

Therefore, our final answer is 1100.

Verilog Code:

SOURCE CODE

```
 module excess3_adder (
  input  wire [3:0] a_ex3,   // input digit A in Excess-3 (should be 3..12 for valid
0..9)
input  wire [3:0] b_ex3,   // input digit B in Excess-3
output reg  [3:0] sum_ex3, // output sum in Excess-3
output reg        cout_dec // decimal carry out (1 when sum >= 10)

);


    // localparams for clarity
       localparam [3:0] THREE = 4'd3;
       localparam [4:0] TEN   = 5'd10;

   reg [3:0] a_bcd, b_bcd;
   reg [4:0] sum_bcd;
   reg [3:0] digit_bcd;

 always @* begin
  // default values
cout_dec  = 1'b0;

       digit_bcd = 4'd0;
      // STEP 1: XS3 → BCD (subtract 3). Assumes valid XS-3 inputs.

       a_bcd = a_ex3 - THREE;
       b_bcd = b_ex3 - THREE;

     // STEP 2: Add the BCD digits (5-bit to hold up to 18)

       sum_bcd = {1'b0, a_bcd} + {1'b0, b_bcd};

   // STEP 3: Correct if >= 10
```

```
  if (sum_bcd >= TEN) begin
     cout_dec  = 1'b1;
    digit_bcd = (sum_bcd - TEN);   // Verilog auto-truncates to 4 bits
    end else begin

         cout_dec  = 1'b0;
        digit_bcd = sum_bcd[3:0];

      end

   // STEP 4: Convert BCD → XS3 (add 3)

      sum_ex3 = digit_bcd + THREE;

   end
   endmodule
```

TEST BENCH

```
timescale 1ns / 1ps
module tb_excess3_adder;

   reg [3:0] a_ex3;
   reg [3:0] b_ex3;

   wire [3:0] sum_ex3;
   wire cout_dec;
   excess3_adder uut (

      .a_ex3(a_ex3),
      .b_ex3(b_ex3),
      .sum_ex3(sum_ex3),
      .cout_dec(cout_dec)

   );
 initial begin

      $dumpfile("dump.vcd");
      $dumpvars(0, tb_excess3_adder);
      $monitor("Time=%0t | A=%b B=%b | Sum=%b Carry=%b", $time, a_ex3,
b_ex3, sum_ex3, cout_dec);
```

```
// Case 1
    a_ex3 = 4'b0011; b_ex3 = 4'b0011;
    #10;

   // Case 2
    a_ex3 = 4'b0110; b_ex3 = 4'b0111;
    #10;

   // Case 3
    a_ex3 = 4'b1011; b_ex3 = 4'b1000;
   #10;
   // Case 4
   a_ex3 = 4'b1100; b_ex3 = 4'b1100;
  #10;

  $finish;

end
endmodule
```

Code Explanation:

1. Purpose of the Module

   Implements addition of two decimal digits encoded in Excess-3 (XS-3).
   Produces the result in XS-3 format and generates a decimal carry-out.

2. Inputs and Outputs

   a_ex3, b_ex3 → 4-bit XS-3 encoded digits (0011 to 1100).
   sum_ex3 → XS-3 encoded sum.
   cout_dec → Carry output when sum ≥ 10.

3. Step 1: XS-3 to BCD Conversion

   XS-3 cannot be added directly, so both inputs are converted to BCD.
   Done by subtracting 3 from each input:
   a_bcd = a_ex3 – 3
   b_bcd = b_ex3 - 3.

4. Step 2: BCD Addition

BCD digits are added using a 5-bit register (sum_bcd).
5 bits are used because 9 + 9 = 18, which requires 5 bits.

5. Step 3: BCD Correction

If sum_bcd >= 10:
Set cout_dec = 1.
Correct the digit by subtracting 10.
Else:
No correction needed; carry remains 0.

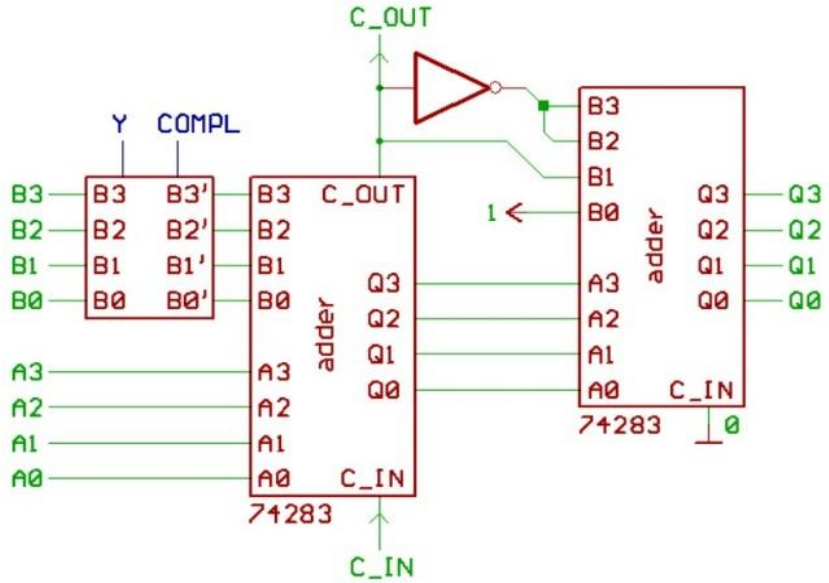
6. Step 4: Convert Back to Excess-3
Corrected BCD digit is re-encoded into XS-3 by adding 3:
sum_ex3 = digit_bcd + 3.

7. Design Characteristics

Fully combinational (updates instantly when inputs change).
Matches the theoretical XS-3 addition process.
Suitable for multi-digit extensions.

Outputs:

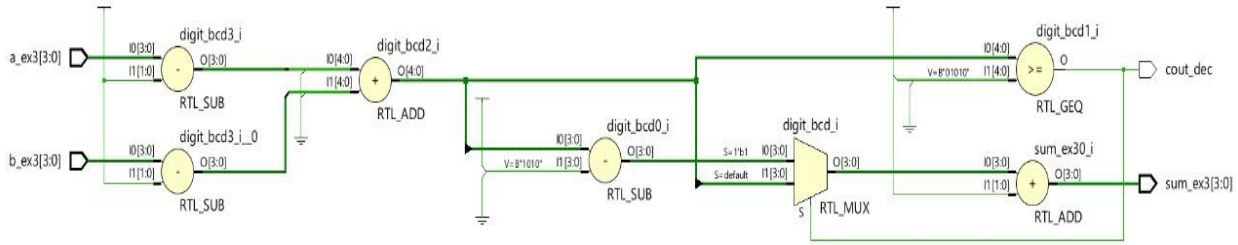
Circuit Diagram:



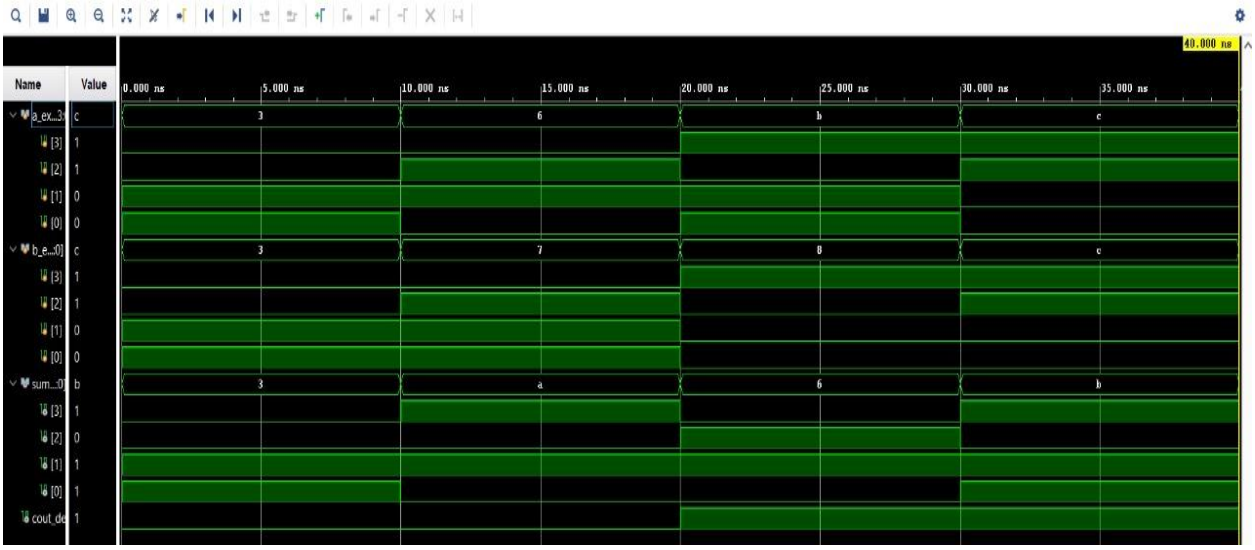
Truth Table:

Decimal Digit (D)	Excess-3 Code (D + 3)	Binary
0	3	0011
1	4	0100
2	5	0101
3	6	0110
4	7	0111
5	8	1000
6	9	1001
7	10	1010
8	11	1011
9	12	1100

Schematic Diagram:



Output Graph:



Result: From above we have observed and verified the outputs along with the working principle for Excess-3 Adder.

References:

1] M. M. Mano and M. D. Ciletti, Digital Design, 5th ed. Pearson, 2013.

2] T. L. Floyd, Digital Fundamentals, 11th ed. Pearson, 2015.

3] M. M. Mano, Digital Logic and Computer Design. Prentice Hall, 1995.

4] NPTEL, "Digital Circuits – Number Systems and Codes," IIT Kharagpur. Available: https://nptel.ac.in

5] All About Circuits, "Excess-3 Code and Decimal Adders." Available: https://www.allaboutcircuits.com

6] GeeksforGeeks, "Excess-3 Code: Definition and Conversion." Available: https://www.geeksforgeeks.org